

Esse é o Dockerfile que define como o container web do seu projeto é construído. Vou explicar linha por linha, de um jeito simples:

FROM php:8.2-apache

Essa é a **imagem base** — o "ponto de partida" do container. Ela já vem pronta com:

- **PHP 8.2** instalado
- **Apache** (o servidor web) instalado e configurado para rodar PHP

Pense nisso como pegar um computador que já vem com o sistema operacional e os programas básicos instalados, em vez de montar tudo do zero.

RUN docker-php-ext-install pdo pdo_mysql mysqli

Isso **instala extensões do PHP** que permitem que seu código converse com bancos de dados MySQL:

- pdo e pdo_mysql — uma forma "moderna" de conectar no banco (não é a que você está usando no cadastrar.php/consulta.php, mas é comum ter instalada)
- mysqli — é a extensão que seus arquivos cadastrar.php e consulta.php realmente usam (funções como mysqli_connect, mysqli_query, etc.)

Sem essa linha, o PHP nem saberia o que fazer com mysqli_connect() — daria erro de função não encontrada.

RUN a2enmod rewrite

Isso **ativa o módulo "rewrite" do Apache**, que permite criar URLs mais amigáveis (ex: transformar site.com/produto/123 em vez de site.com/produto.php?id=123). No seu projeto atual talvez não esteja em uso ainda, mas é comum deixar ativado por padrão, caso precise no futuro (ex: para um sistema de login com rotas).

WORKDIR /var/www/html

Define a **pasta de trabalho padrão** dentro do container. É como dizer "a partir de agora, quando eu rodar comandos aqui dentro, considere que estou nessa pasta". Também é a pasta onde o Apache procura os arquivos PHP para servir — por isso seu docker-compose.yml monta ./src:/var/www/html: seus arquivos locais aparecem exatamente nessa pasta dentro do container.

RUN chown -R www-data:www-data /var/www/html

Isso **muda o dono dos arquivos** dessa pasta para o usuário www-data, que é o usuário que o Apache usa para rodar. Isso evita problemas de permissão (tipo o Apache não conseguir ler ou escrever nos arquivos). O -R significa "recursivo" — aplica em todos os arquivos e subpastas.

EXPOSE 80

Isso **documenta** que o container vai escutar na porta 80 (a porta padrão do Apache/HTTP). É mais uma informação/declaração do que uma ação real — quem de fato "abre" a porta para você acessar de fora é o ports: "8080:80" que está no seu docker-compose.yml.

Resumindo em uma frase: esse Dockerfile pega um PHP com Apache pronto, instala o que falta para conversar com MySQL, ajusta permissões, e deixa preparado para servir seus arquivos .php na porta 80 (que você acessa de fora pela porta 8080, conforme configurado no compose).

Faz sentido? Se quiser, posso desenhar um diagrama mostrando como esses containers (web, db, phpmyadmin) se conectam entre si.